EAST WEST UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE DESIGN DOCUMENT

Project Title

scholar-agent: An Autonomous Scholarship and PhD Application Platform

Course: CSE 412 (Software Engineering)

Submitted By

Student Name	Student ID
Md. Tamim Hasan Saykat	2022-1-60-289
Md. Mahamudur Rahman Maharaz	2022-3-60-182
Naima Islam	2022-3-60-215

Student Contribution

Student Name	Contribution
Md. Tamim Hasan Saykat	34%
Md. Mahamudur Rahman Maharaz	33%
Naima Islam	33%

Submitted To

Yasin Sazid
Department of Computer Science and Engineering, East West University

Design Methodology

Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, Chapter 13 (Architectural Design) and Chapter 14 (Component-Level Design).

1 Introduction

1.1 Project Overview

scholar-agent is a web platform for finding, tracking, and completing funded study opportunities (scholarships and PhD positions) from a single account. It has two cooperating parts. A **discovery engine** searches the live web, extracts structured opportunity records, ranks them against the user's CV, and drafts application materials. A **system of record**, which is the web application itself, stores accounts, tracks each application through a status pipeline, manages professor outreach, and hosts a writing practice module.

The split between these two parts is deliberate. The discovery engine answers the question *"what opportunities exist?"* and writes to a shared knowledge base built from a graph store and a vector store. The system of record answers *"what is this user doing about them?"* and writes to a per-user relational database. This separation is the central design decision of the system, and the rest of the document refers back to it.

1.2 List of Requirements

1.2.1 Functional Requirements

ID	Requirement
FR-1	A visitor can register and authenticate; all data is private to the account.
FR-2	A user uploads or pastes a CV; the system extracts a structured profile.
FR-3	The system searches the web from free-text topics and extracts structured opportunity records.
FR-4	Extracted opportunities are persisted to the knowledge base and de-duplicated by content identity.
FR-5	A user browses and filters the catalog and saves opportunities into a personal pipeline.
FR-6	The system ranks opportunities against a profile and returns a score with a rationale.
FR-7	A user tracks each application through a status pipeline with a requirement checklist.
FR-8	The system drafts a grounded cold email and Statement of Purpose per application, and persists them.
FR-9	A user maintains a professor-outreach record with a message thread and follow-up dates.
FR-10	The system exports all deadlines and follow-ups as an importable calendar (.ics).
FR-11	A user keeps a watchlist of standing interests surfed automatically each day.
FR-12	A user submits writing and receives criterion-level band scoring with feedback.

1.2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	Passwords stored only as Argon2 hashes; sessions use signed JWTs; every query is scoped by `user_id`.
NFR-2	Portability	Runs with zero external services in development (SQLite, single provider); scales to Postgres and a multi-provider pool without code change.
NFR-3	Resilience	Inference degrades across a provider pool: on rate-limit or outage a call rolls to the next provider, and finally to a local model.
NFR-4	Cost	Each research pass is bounded (paced request rate, capped page counts, per-role model routing) to fit free API tiers.
NFR-5	Maintainability	Build-free frontend; typed, layered backend with an automated test suite.
NFR-6	Extensibility	New providers, agents, and API resources are added without modifying existing components (§3.3).

1.3 Technology Stack

Layer	Technology	Role
Language	Python 3.12	Backend and agents
Web framework	FastAPI (ASGI)	HTTP API, dependency injection
Validation	Pydantic v2	Domain schemas, request/response contracts
ORM	SQLAlchemy 2.0	Relational persistence
Relational DB	SQLite / PostgreSQL	System of record
Graph store	Neo4j	Opportunity relationships
Vector store	Qdrant	Semantic search over opportunities
Agent orchestration	LangGraph	Discovery state machine
Retrieval	fastembed (BGE + MiniLM)	Local embeddings and reranking
Auth	passlib (Argon2), PyJWT	Hashing, token signing
Frontend	HTML + Alpine.js + Tailwind	Build-free reactive UI
Model serving	Groq, Gemini, Ollama (pool)	LLM inference with failover

2 Architectural Design

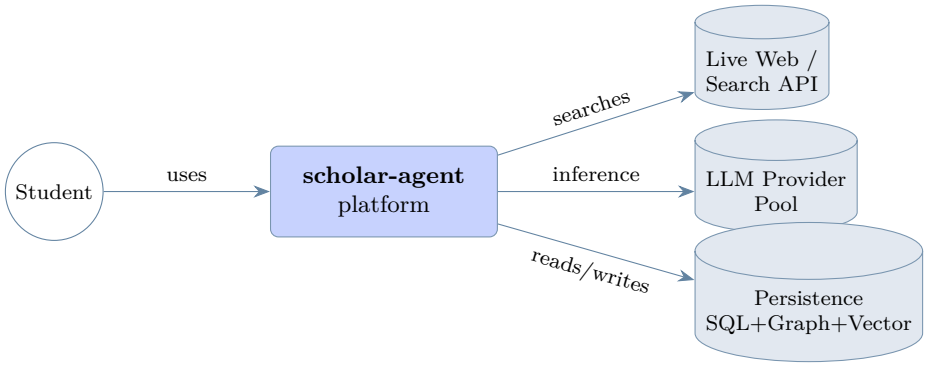
This section follows Pressman Chapter 13: an architectural context diagram at two levels (§2.1), a top-level component view (§2.2), and component instantiation with elaboration (§2.3).

2.1 Architectural Context Diagram

The architectural context diagram models how the system connects to entities outside its boundary: superordinate systems that use it, subordinate systems it uses, and actors (Pressman §13.3.1).

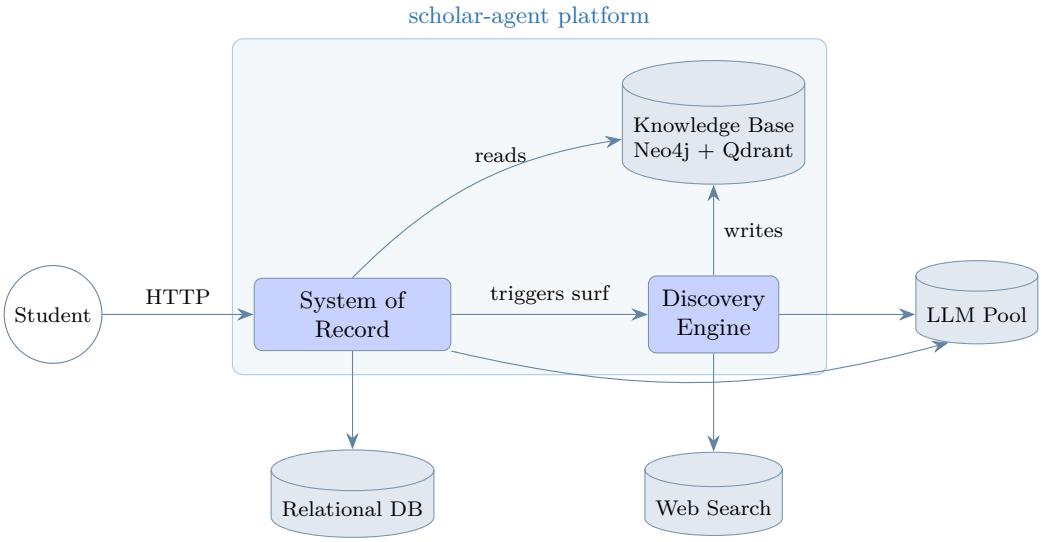
2.1.1 Level 0: System in Context

At Level 0 the platform is one component. The student uses it; it depends on three external subordinate systems: the web, the LLM provider pool, and the persistence stores.



2.1.2 Level 1: Internal Subsystems in Context

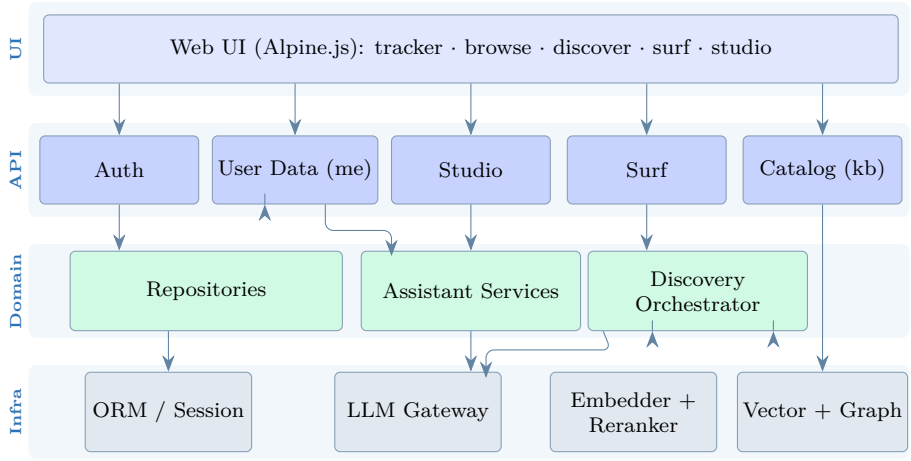
At Level 1 the platform opens into its two subsystems and the shared knowledge base, showing which external entities each subsystem contacts.



The System of Record is the only subsystem the student touches. It owns the relational database and reads (never writes) the knowledge base. The Discovery Engine is the only writer of the knowledge base and the only subsystem that reaches the outside web; it is invoked by the System of Record, not the user. The knowledge base is the single decoupling surface between the two.

2.2 Top-Level Component Diagram

The macroscopic view decomposes the platform into components (Pressman §13.4 to 13.5). Each box is a package boundary in the codebase.



2.3 Component Instantiation and Elaboration

Each component is given a responsibility and interface, then its operation is described.

Web UI.
Static HTML pages driven by small Alpine.js components. Each fetches JSON with a bearer token from browser storage. No build step; Tailwind and Alpine load from a CDN. The UI holds no business rules; it only renders state and issues requests.

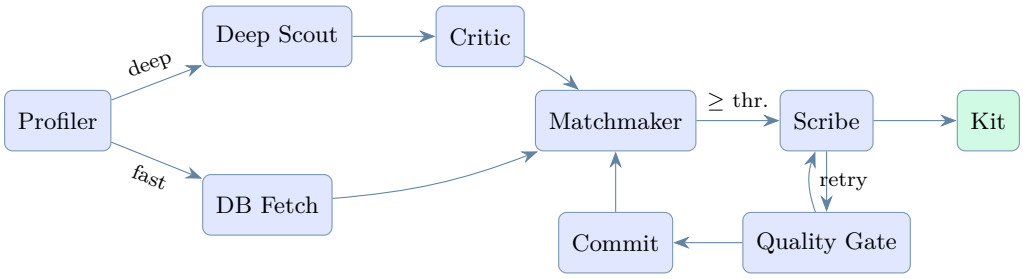
API Controllers.
FastAPI routers, one per resource family: `me` owns the user's data (profile, applications, professors, watchlist, calendar, drafting), `kb` exposes the read-only catalog, `surf` triggers discovery, `studio` scores writing. Every controller depends on one `CurrentUser` dependency that decodes the JWT. This is the single point where identity enters the system, which is what makes per-user isolation enforceable (NFR-1).

Domain / Services.
Repositories translate controller intent into ORM queries always filtered by `user_id`. The Discovery Orchestrator compiles and runs the agent state machine. Assistant Services combine stored data with inference: drafting materials, building the calendar, scoring writing.

Infrastructure.
The LLM Gateway exposes one surface (`complete`, `structured`) over an ordered provider pool; on failure it rolls to the next provider, finally to a local model (NFR-3). The Vector and Graph adapters wrap Qdrant and Neo4j. The Embedder/Reranker produces vectors and re-scores candidates locally, so retrieval quality does not depend on a paid API.

2.3.1 Elaboration of the Discovery Orchestrator

The orchestrator is the most behaviour-rich component. It is a LangGraph state machine over a shared `PipelineState`. It has two entry paths, a deep web search and a fast database lookup, that converge at matching and then enter a Scribe/Quality-Gate refinement loop.



The Profiler turns a CV into a structured profile and routes the run. The Deep Scout searches the web and extracts opportunity records; the Critic fills missing deadlines. The Matchmaker scores each opportunity against the profile and gates on a threshold. The Scribe drafts materials; the Quality Gate verifies every claim is grounded in source text and loops back until it passes. This bounded loop is what prevents hallucinated output.

3 Component-Level Design

This section follows Pressman Chapter 14: the three class-elaboration steps (§3.1), the derived class diagram (§3.2), the design principles (§3.3), and the database design (§3.4).

3.1 Elaboration of Design Components

3.1.1 Step 1: Problem-Domain Design Classes

These model the vocabulary of the problem itself and are technology-independent.

Design class	Responsibility
`User`	Account holder; owns all other records.
`StudentProfile`	Structured applicant record derived from a CV.
`Opportunity`	A funded position: title, institution, funding, deadline, supervisor.
`SavedApplication`	A user's tracking record: status, checklist, drafted documents.
`ProfessorContact`	An outreach target with a message thread and follow-up schedule.
`WatchlistItem`	A standing search interest surfed automatically.
`MatchResult`	Score and rationale relating a profile to an opportunity.
`Artifact` (`ColdEmail`, `SOPDraft`)	A generated application document.
`WritingFeedback`	Criterion-level band scoring of a piece of writing.

3.1.2 Step 2: Infrastructure-Domain Design Classes

These provide the technical services (persistence, network, security, mediation) the problem-domain classes need.

Design class	Domain	Responsibility
`*Repository`	Persistence	Scoped CRUD, always filtered by `user_id`.
`Session`	Persistence	Unit-of-work / transaction boundary.
`VectorStore`	Persistence	Upsert and semantic search over opportunities.
`GraphStore`	Persistence	Relationship storage and freshness pruning.
`Embedder/Reranker`	Compute	Local vector generation and re-scoring.
`LLMClient`	Network	Provider-agnostic inference with failover.
`ProviderConfig`	Network	One endpoint's URL, key, per-role model map.
`AuthService`	Security	Argon2 hashing, JWT issue/verify.
`CurrentUser`	Mediation	Resolves the authenticated user per request.

3.1.3 Step 3b: Component Interfaces

Step 3b identifies the interface (message set) each component exposes. Defining them as abstract contracts is what enables the substitution and inversion in §3.3.

Interface	Operations	Realised by
`InferenceProvider`	`complete`, `structured`	`LLMClient` over any `ProviderConfig`
`Repository<T>`	`list`, `get`, `create`, `update`, `delete`	each concrete repository
`VectorIndex`	`upsert`, `hybrid_search`, `scroll_all`, `fetch_by_id`	Qdrant adapter
`AgentNode`	`run(state)` → state	each of the six agents
`Assistant`	`draft_application`, `draft_email`, `build_calendar`, `score_writing`	Assistant services

At the system boundary these interfaces surface as a REST API. The table below lists the concrete endpoints each controller exposes, so the abstract interfaces above can be traced to the running system.

Resource	Endpoints
Auth	`POST /auth/signup` · `POST /auth/login` · `GET /auth/me`
Profile	`GET /me/profile` · `PUT /me/profile`
Applications	`GET|POST /me/applications` · `GET|PATCH|DELETE /me/applications/{id}` · `POST /me/applications/{id}/draft`
Professors	`GET|POST /me/professors` · `GET|PATCH|DELETE /me/professors/{id}` · `POST /me/professors/{id}/draft_email`
Watchlist	`GET|POST /me/watchlist` · `DELETE /me/watchlist/{id}`
Calendar	`GET /me/calendar.ics` (iCalendar export of deadlines and follow-ups)
Catalog	`GET /kb/opportunities` (read-only, deadline-sorted, expired excluded)
Discovery	`POST /pipeline/stream` (SSE progress) · `POST /maintenance/surf` · `POST /maintenance/sweep`
Studio	`POST /studio/feedback` (band-scored writing assessment)

3.2 Class Diagram

The classes below are derived directly from the Step 1 and Step 2 tables. Problem-domain classes carry key attributes; infrastructure is shown through the Step 3b interfaces.

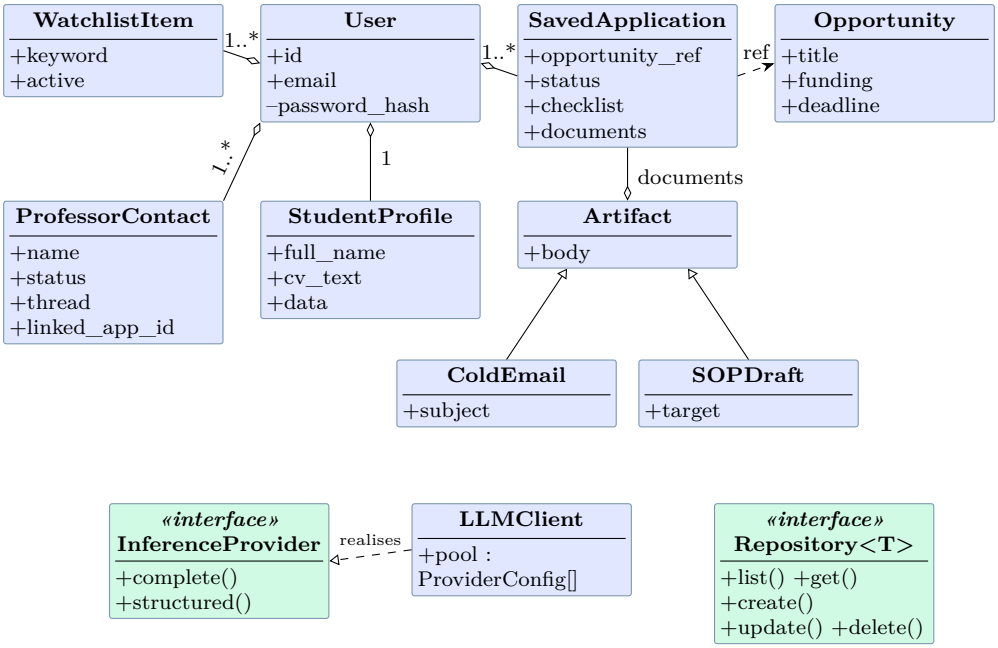


Figure 1: Design class diagram. Open diamonds mark aggregation/ownership; hollow triangles mark generalisation and interface realisation; dashed arrows mark dependency. `User` owns every per-user record; `Artifact` generalises the two generated document types; `LLMClient` realises `InferenceProvider`.

3.3 Application of Pressman's Design Principles

Pressman (Ch. 14, "Designing Class-Based Components") prescribes the four SOLID principles. Each is applied concretely below.

Open Closed Principle (OCP).
The `InferenceProvider` interface lets new model back-ends be added without modifying callers. The provider pool is data-driven (`providers.json`): adding Cerebras or a local model is a config entry, not a code change. The agent pipeline is open for extension: a new `AgentNode` is registered on the state graph while existing nodes stay closed. New API resources are new routers, not edits to existing controllers.

Liskov Substitution Principle (LSP).
Every `ProviderConfig` is substitutable for every other behind `InferenceProvider`: the failover loop swaps a rate-limited Groq provider for Gemini, then local Ollama, with no change to the contract the agents observe. `ColdEmail` and `SOPDraft` are usable wherever an `Artifact` is expected, for instance when appending to an application's `documents` list.

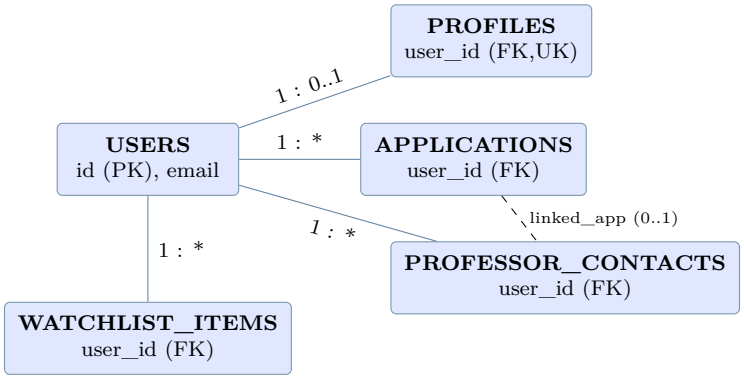
Dependency Inversion Principle (DIP).
Policy does not depend on detail; both depend on abstractions. The agents depend on the `InferenceProvider` and `VectorIndex` interfaces, not on Groq's SDK or Qdrant's client. Controllers depend on repository interfaces, not on SQLAlchemy internals. Concrete implementations are injected through FastAPI's dependency system and cached factories, so source dependencies point toward abstractions.

Interface Segregation Principle (ISP).
Interfaces are narrow. `InferenceProvider` exposes only `complete`/`structured`; `VectorIndex` exposes only retrieval; each repository exposes only its five CRUD operations. A catalog-reading client depends on `VectorIndex` alone and is never forced to know about the relational repositories. Controllers are split by resource family so none carries operations its clients do not use.

3.4 Database Design

The relational schema is the system of record. Every user-owned table carries a `user_id` foreign key to `users` with cascade delete, so per-user isolation is structural (NFR-1). Small bounded collections such as a checklist, a message thread, or a set of drafted documents are stored as embedded JSON columns rather than child tables. They are always fetched with their parent and never queried on their own, so a separate table would add joins for no benefit.

3.4.1 Entity Relationship (ER) Diagram



3.4.2 Table Schemas (key columns)

Column	Type / Key	Notes
users		
`id`	INTEGER PK	Identity root.
`email`	VARCHAR(320) UNIQUE	Login handle.
`password_hash`	VARCHAR(255)	Argon2 hash only; the raw password is never stored.
profiles		
`user_id`	FK UNIQUE, CASCADE	One profile per user.
`data`	JSON	Structured `StudentProfile` extracted from the CV.
applications		
`user_id`	FK INDEX, CASCADE	Owner.
`opportunity_ref`	VARCHAR(32) null	KB opportunity id, or null if added manually.
`status`	VARCHAR(40) INDEX	Pipeline stage: interested, preparing, applied, interview, offer, rejected, or waitlisted.
`checklist`	JSON	`[{label, done}]`.
`documents`	JSON	`[{type,title,body,created_at}]`; server-generated drafts.
professor_contacts		
`status`	VARCHAR(40) INDEX	Outreach stage: to_contact, emailed, replied, in_conversation, meeting, or closed.
`linked_application_id`	INTEGER null	Soft link to an application.
`next_followup_at`	VARCHAR(32) null	Feeds the calendar export.
`thread`	JSON	`[{direction,subject,body,at}]`.
watchlist_items		
`keyword`	VARCHAR(300)	Standing search topic.
`last_surfed_at`	VARCHAR(32) null	Drives the oldest-first daily rotation.

3.4.3 Relationship Mappings

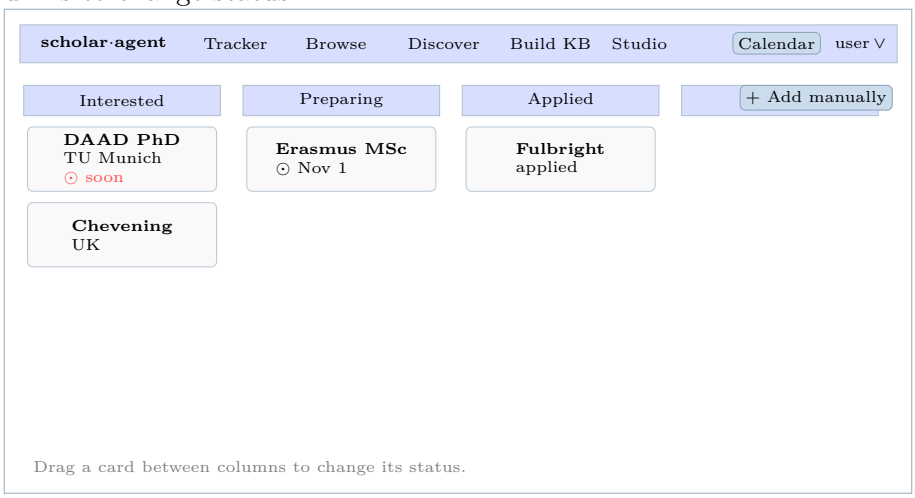
- **users** → **profiles**: one-to-one (unique `user_id`), cascade delete.
- **users** → **applications / professor_contacts / watchlist_items**: one-to-many with cascade delete, so removing an account removes all of its data.
- **applications** ⇢ **professor_contacts**: optional many-to-one soft link (`linked_application_id`), not a hard FK, so outreach can exist independently.
- **applications** ⇢ **knowledge base**: a cross-store reference by content id (`opportunity_ref`), deliberately not a FK, preserving the discovery/record decoupling. The knowledge-base entities live in Neo4j/Qdrant, outside this relational model.

4 User Interface Design

The interface is a small set of single-purpose pages sharing one top navigation bar and a dark, glass-panel theme. The wireframes below are low-fidelity (boxes and labels only), so they show layout and interaction without committing to final visual styling. Each screen is shown separately.

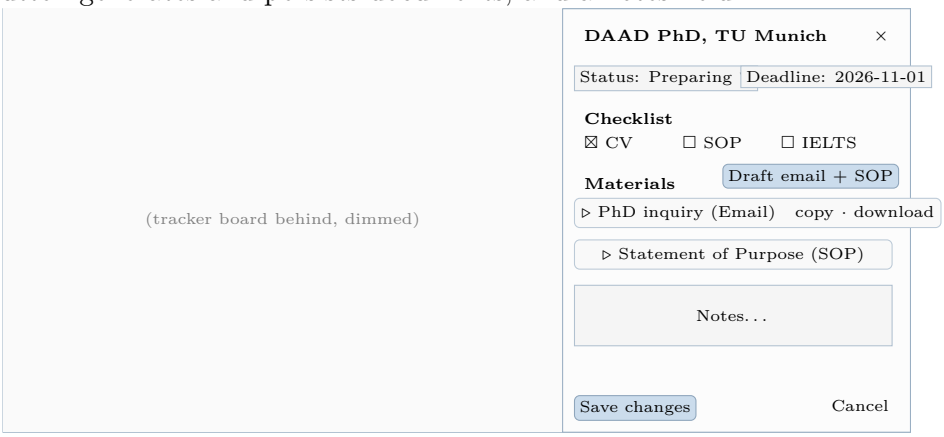
4.1 Wireframe 1: Application Tracker (home)

The home screen is a Kanban board with one column per pipeline stage. A fixed top bar carries the brand, page navigation, a calendar-export action, and the account menu. Cards are dragged between columns to change status.



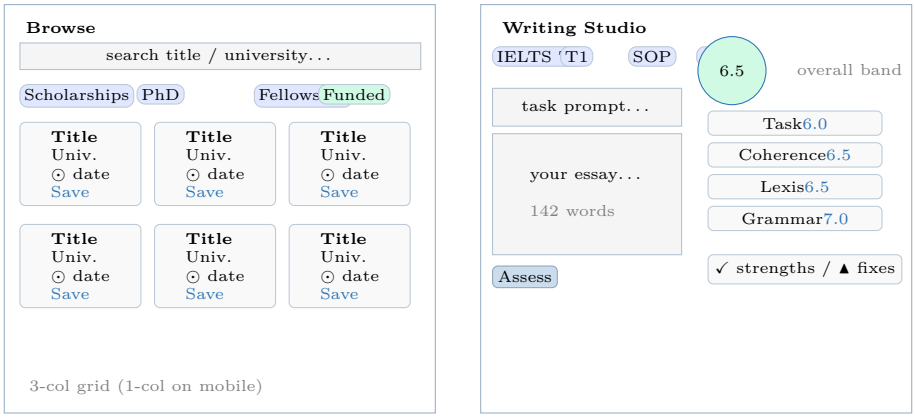
4.2 Wireframe 2: Application Detail Drawer

Clicking a card slides in a right-hand drawer (an in-place sub-state, not a new page). It holds the status and deadline controls, a requirement checklist, a *Materials* section whose "Draft email + SOP" button generates and persists documents, and a notes field.



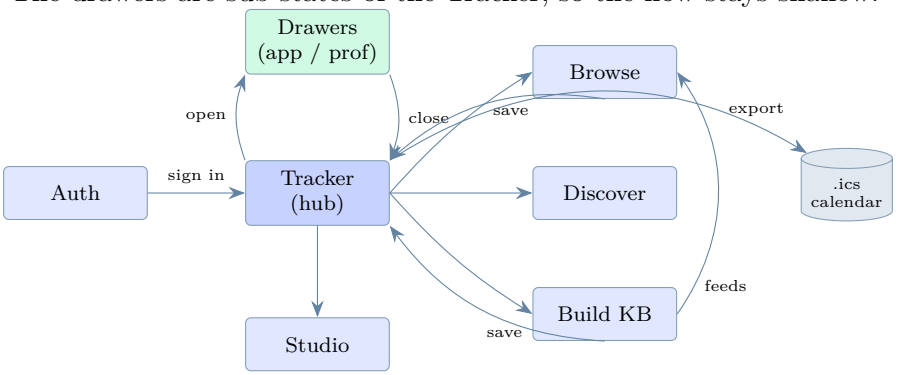
4.3 Wireframe 3: Browse Catalog and Writing Studio

Two further screens are shown together. Browse is a filtered card grid over the knowledge base; the Studio is a two-panel practice screen (input left, scored feedback right).



4.4 Navigation Flow

Authentication is the single entry gate: every other screen needs a valid session and otherwise redirects to sign-in. The Tracker is the hub: the feature pages branch off it and most actions return to it. The drawers are sub-states of the Tracker, so the flow stays shallow.



End of Software Design Document.